ISTQB® Certified Tester

Advanced Level — **Security Tester** (CT-SEC, Syllabus 2016)

CHAPTER 5

Testing Security Mechanisms

Complete Study Guide

Chapter fact	Value
Syllabus sections covered	5.1 System Hardening · 5.2 Authentication and Authorization · 5.3 Encryption · 5.4 Firewalls and Network Zones · 5.5 Intrusion Detection · 5.6 Malware Scanning · 5.7 Data Obfuscation · 5.8 Training
Learning objectives	16 examinable (AS-5.1.1 – AS-5.8.2) · K2 = 8 · K3 = 8 — one “understand” and one “test” objective per mechanism
Recommended teaching time	240 minutes — **the longest chapter** — **12 points**, 3.0 points per teaching hour
Questions in the exam	**8 of 45 questions · 12 of 80 points. K2 = 4** (drawn from the 8 K2 objectives) · **K3 = 4** (drawn from the 8 K3 objectives). In the Sample Exam each mechanism appeared once
Exam format	45 questions · **80 points** · **120 minutes** (150 with the 25% non-native-language allowance) · pass mark 65% = **52 points** · K2 = 1 pt · K3 = 2 pts · K4 = 3 pts
Keywords	anti-malware · authentication · authorization · demilitarized zone · encryption · firewall · hashing · insider threat · intrusion detection system · malware · malware scanning · network zone · pharming · phishing · salting · system hardening · vulnerability scanner
Official sample-exam questions	**#25 – #32** (2 + 1 + 2 + 1 + 2 + 1 + 2 + 1 points)

Built exclusively from the ISTQB® Advanced Level Security Tester syllabus (2016), the official Sample Exam (questions and answers, GA March 2016) and the ISTQB® exam structure tables.

Study this single file to master Chapter 5.

Contents

Section 1 — Syllabus Explanation....4
Chapter framing — mechanisms are layers, not solutions....4
AS-5.1.1 (K2) — Understanding System Hardening....5
AS-5.1.2 (K3) — Testing the Effectiveness of System Hardening....5
AS-5.2.1 (K2) — Authentication and Authorization....6
AS-5.2.2 (K3) — Testing Authentication and Authorization....6
AS-5.3.1 (K2) — Understanding Encryption....7
AS-5.3.2 (K3) — Testing Common Encryption Mechanisms....7
AS-5.4.1 (K2) — Understanding Firewalls and Network Zones....8
AS-5.4.2 (K3) — Testing Firewall Effectiveness....9
AS-5.5.1 (K2) — Understanding Intrusion Detection Tools....9
AS-5.5.2 (K3) — Testing Intrusion Detection Tools....10
AS-5.6.1 (K2) — Understanding Malware Scanning Tools....10
AS-5.6.2 (K3) — Testing Malware Scanning Tools....10
AS-5.7.1 (K2) — Understanding Data Obfuscation....11
AS-5.7.2 (K3) — Testing Data Obfuscation Approaches....11
AS-5.8.1 (K2) — The Importance of Security Training....12
AS-5.8.2 (K3) — Testing the Effectiveness of Security Training....12
Section 2 — Key Concepts....13
2.1 The storyline of Chapter 5....13
2.2 The eight mechanisms....13
2.3 Fuzzy points clarified....17
2.4 Chapter 5 keyword glossary....18
Section 3 — Tables and Summaries....19
Section 4 — Demo Questions with Answers....20
Part A — The 8 official Chapter 5 questions....20
Part B — 24 additional syllabus-derived questions....22
Most Important Questions From This Chapter....28
Section 5 — Exam Tricks....29
5.1 Frequently tested topics (evidence-based)....29
5.2 Common distractors observed....29
5.3 Keyword recognition (map words → answers)....29
5.4 Typical traps....30
5.5 How to eliminate wrong answers....30
5.6 Scenario-solving shortcuts....30
5.7 Common candidate mistakes....31
5.8 Mnemonics that carry the whole chapter....31
5.9 Chapter 5 mind map....31
Section 6 — Revision Notes....32
6.1 One-page chapter summary....32
6.2 Key definitions to memorize....32
6.3 High-priority concepts (study first)....32
6.4 Last-minute revision sheet....32
6.5 Memorization checklist....33
6.6 Exam-day quick review....33

In Word, right-click the table of contents and choose "Update Field" → "Update entire table" to refresh the page numbers.

How to use this guide

Section 1 walks the eight mechanisms, each with its K2 concept and K3 effectiveness test — read it once. **Section 2** explains every key concept from the ground up — plain words, how it works, a worked example, the exam angle, and what it is easily confused with — and ends with the chapter's **fuzzy points clarified**. If the syllabus wording in Section 1 feels dense, read the matching Section 2 card first. **Section 3**, **table 3.1 (mechanism → weakness → test)**, is the single most useful table in this guide. **Section 4** is the question bank, **Section 5** exam tactics, **Section 6** is the revision sheet for the night before.

What makes Chapter 5 unusual: it is the **longest chapter (240 minutes)** for only **12 points** — **3.0 points per hour**, the joint-lowest return. Sixteen objectives feed eight questions, so on any paper about half of the content is not examined. **Breadth beats depth here.**

The rule that decides it: for every mechanism, **know the weakness the syllabus names, because the test exploits it** — anti-malware only knows the known (EICAR, pilot first), signature IDS misses the unknown (evasion), behaviour IDS drifts (feed it malicious traffic), obfuscation is reversible (brute force, decompilers), humans are the weakest link (social engineering, sticky notes).

Suggested schedule: Day 1 — §5.1–5.4. Day 2 — §5.5–5.8 and Section 3. Day 3 — Section 4 and Section 5. Day 4 onwards — redo the questions you missed and re-read Section 2's fuzzy points. Night before — Section 6.

Section 1 — Syllabus Explanation

Chapter 5, **Testing Security Mechanisms**, covers **eight mechanisms** — system hardening, authentication and authorization, encryption, firewalls and network zones, intrusion detection, malware scanning, data obfuscation, and training — each with a **K2 "understand" objective** and a **K3 "test the effectiveness" objective**: **16 learning objectives** in all. The Exam Structure Table allocates **8 questions** and **12 of the 80 points**.

Teaching time is **240 minutes — the longest chapter in the syllabus** — for **3.0 points per teaching hour**, the joint-lowest return. The questions are drawn as **four K2 (1 point) from the eight "understand" objectives** and **four K3 (2 points) from the eight "test" objectives**. In the official Sample Exam, **each mechanism appeared exactly once**, alternating K3 and K2 (Q25 to Q32). Learn every mechanism to the same depth, but **do not over-invest**: the content is broad and the marks are thin.

Mechanism	K2 — understand	K3 — test effectiveness	Sample #
5.1 System hardening	AS-5.1.1	AS-5.1.2	#25 (K3, 2 pts)
5.2 Authentication and authorization	AS-5.2.1	AS-5.2.2	#26 (K2, 1 pt)
5.3 Encryption	AS-5.3.1	AS-5.3.2	#27 (K3, 2 pts)
5.4 Firewalls and network zones	AS-5.4.1	AS-5.4.2	#28 (K2, 1 pt)
5.5 Intrusion detection	AS-5.5.1	AS-5.5.2	#29 (K3, 2 pts)
5.6 Malware scanning	AS-5.6.1	AS-5.6.2	#30 (K2, 1 pt)
5.7 Data obfuscation	AS-5.7.1	AS-5.7.2	#31 (K3, 2 pts)
5.8 Training	AS-5.8.1	AS-5.8.2	#32 (K2, 1 pt)
Chapter 5 total	**4 questions × 1**	**4 questions × 2**	**8 questions · 12 points**

Exam allocation you must know

Chapter 5 = **8 questions / 12 points** from **16 objectives**; Sample Exam **#25 to #32**. Every K3 objective has the same verb — **"demonstrate how to test the effectiveness of …"** — and every K2 has the same shape — **"understand the concept of … and how it is applied"**. A K3 question usually offers **one test technique the syllabus names for that mechanism** among three plausible activities that are not tests of it.

The one thing that decides this chapter

For each mechanism, know its weakness — the test exploits the weakness. Hardening: defaults and unnecessary services → **audit configuration, run a vulnerability scanner**. Authentication: weak passwords and unfiltered input → **brute force, dictionary, injection, URL/URI tampering**. Encryption: small keys, wrong modes, bad randomness, weak configuration → **key size, modes, certificates, code review, TLS configuration**. Firewalls: complexity → **port scanning, malformed packets, fragmentation**. IDS: signatures miss the unknown, behaviour drifts → **evasion encodings, malicious traffic to poison the profile**. Anti-malware: **only knows what it knows** → **EICAR**, test before deploying. Obfuscation: reversible → **brute force, decompilers**. Training: humans → **social engineering, sticky notes, password audits**.

Chapter framing — mechanisms are layers, not solutions

Each mechanism can be applied through **tools and infrastructure** or **manual effort**. **None of these mechanisms alone is sufficient in most cases** to secure information; **each has its own advantages and disadvantages**. Testers need to understand **the nuances of each line of defense** to design tests that **verify and validate their effectiveness**, and advanced testers should **design a test architecture that provides a framework for continuous security testing**.

AS-5.1.1 (K2) — Understanding System Hardening

Original syllabus statement

Understand the concept of system hardening and its role in enhancing security.

Systems grow more complex, so **their attack surface is continuously growing**. Vulnerabilities come from three sources:

Source	Syllabus term	Example
Design errors	**By design** vulnerabilities	Missing authorization layer
Source code defects	**By construction** vulnerabilities	Buffer overflow
Lack of rigor in configuration	**By configuration** vulnerabilities	Default passwords, open ports

System hardening is the step-by-step process of reducing the attack surface by applying a security policy and different layers of protection. The main objective is to secure the system and reduce the risks of compromising security. It can be applied to **a component, a product/application, a system, or a system of systems**.

Organizational and technical defenses to apply

Defense	Why (syllabus)
Remove unnecessary **software**	Can contain defects
Remove unnecessary **libraries and developer tools**	Can contain defects
Remove unnecessary **accounts/logins**	Attack vectors
Remove unnecessary **applications and network services**	Defects; attack vectors
Remove unnecessary **peripherals and hardware ports** (USB, card readers)	—
Promptly patch and install updates (e.g. **automatic updates**)	—
Update configurations	—
Follow coding rules	Avoid **by-construction** vulnerabilities
Configure a remote logging server (e.g. **rsyslog**)	An attacker can delete logs **only on the compromised machine, not on the remote log server**

Security mechanisms that hardening should use: strong authentication and efficient **authorization** (only the rights needed, by role) · **encryption** (communication and local storage) · **firewalls** (personal, system, web application) and **security zones** (e.g. a **sandbox**) · **IDS** · **anti-malware/anti-spyware** · **obfuscation** of data and applications (against reverse-engineering).

The balance

Rules must be applied **at the right level and balanced with usability**. **At the extreme, protections are disabled because they block the productivity of the company** — over-hardening defeats itself.

AS-5.1.2 (K3) — Testing the Effectiveness of System Hardening

Original syllabus statement

Demonstrate how to test the effectiveness of common system hardening mechanisms.

Tests depend on **the nature of the system, the sensitivity of assets and the identified threats**. Hardening **restricts access to the right roles, opens only needed services, and monitors application updates** — so tests should show hardening is **working,**

applied in the right places, and applied in the right ways. **Also test for protections that are too restrictive** and excessive in view of the risks.

- Some tests are **review- or audit-based**; others test **the ability of user groups to perform actions or access data**.
- **Audit database and application server configuration** to verify **default passwords have been changed**.
- **Audit system configuration** to identify **unnecessary services and network ports**.
- **Verify components, libraries and application versions** are **not obsolete and vulnerable**.
- Run a **vulnerability scanner** to ease vulnerability assessment, especially in **complex (e.g. multi-site)** systems.
- Use **static analysis tools** to detect **violations of coding rules** (construction vulnerabilities); **security-oriented analyzers** are particularly useful.

How the official exam used this objective (K3, 2 points)

Sample Q25: procedure to ensure hardening mechanisms work as expected? **Closely monitor security performance reports and metrics to determine if the correct level of access and authentication is achieved.** Rejected: auditing **strong authentication** (just **one aspect** of hardening); seeking **equilibrium** between hardened components (not needed — **more critical areas may warrant better hardening**); enlisting **a known hacker** (**the danger of the hacker not telling you what is found**).

AS-5.2.1 (K2) — Authentication and Authorization

Original syllabus statement

Understand the relationship between authentication and authorization and how they are applied in securing information systems.

	Authentication	Authorization
Question answered	**Login: who is the user? Password: is the user really who he pretends to be?**	**Does the authenticated user have the rights to do this action?** Which **level of access** to resources?
Based on	Verification of **a user identifier and a token**	Rights assigned to the authenticated identity
Implementations	Detecting **weak passwords · one-time passwords (OTP) · fingerprinting · software certificates · certificates in hard tokens**	e.g. can log into a server **but not modify its data**; FTP use **only in a dedicated space**
Architectures	**Local · server · network authentication · Single Sign-On (SSO)**	Role-based levels

The link: a non-authenticated user has no rights or restricted rights (not authorized to manipulate sensitive data). Merchant site example: an anonymous user **can see the product list**; to buy, they **create an account**; the authenticated user **can purchase but cannot perform administrative functions**.

How the official exam used this objective (K2, 1 point)

Sample Q26: key attribute of **authentication**? **It verifies that the user entering the system is legitimate.** Rejected: **correct profile and rights to limited parts** (that is authorization — **not looking at access rights**); **system resources** (not a consideration); **common credentials** (**each individual should have unique credentials**).

AS-5.2.2 (K3) — Testing Authentication and Authorization

Original syllabus statement

Demonstrate how to test the effectiveness of common authentication and authorization mechanisms.

Attackers aim **to steal passwords or bypass systems** to execute unauthorized actions, exploiting **coding errors (lack of input filtering) · old vulnerable library versions · configuration errors (default passwords, default rights) · weak passwords** (the most used is **"123456"**). Password rules **don't help if users aren't diligent**, and must reflect **current good practice** (e.g. **SANS Password Construction Guidelines**).

Tests for authentication and authorization

Test	Example
Brute force and dictionary attacks	Try **"123456", "111111", date of birth, name of pet**…
Exploit lack of input filtering	**Inject SQL** to be **authenticated without any known login/password**
Unauthorized URI / URL	**../../** in an FTP account; **site address/admin** to reach sensitive data
Exploit an unpatched vulnerability	Cause unintended behavior → **gain control** → **privilege escalation**

AS-5.3.1 (K2) — Understanding Encryption

> **Original syllabus statement**
> Understand the concept of encryption and how it is applied in securing information systems.

Encryption is a process of encoding data (plain text) into cypher data (cypher text), using a cryptographic algorithm and secrets, so that only authorized persons can access it by using a decryption mechanism. The secret is **shared and only known to authorized users**. Goal: strong enough that an attacker **who has stolen encrypted data cannot recover the plain text**. **Hashing and salting are methods used during encryption.**

Protects data…	Examples
Stored in a system	**Cyphered passwords in a database**, logical cyphered drive, **whole cyphered hard drive**
During communication	**Cyphered email**, cyphered protocols (**SSL, TLS**)

Protocol family	Key model
Symmetric encryption	**Shared secret key**
Asymmetric encryption	**Private and public key**

*The syllabus says cryptography helps ensure confidentiality, integrity, availability of sensitive assets "and repudiation of manipulating them" — read this as **non-repudiation**.*

AS-5.3.2 (K3) — Testing Common Encryption Mechanisms

> **Original syllabus statement**
> Demonstrate how to test the effectiveness of common encryption mechanisms.

Some mechanisms are **weak because of short or static keys**; others because they are **not implemented with best practices** or **embed coding defects** (e.g. buffer overflow). Tests fall into **three families plus ageing** — the same design / construction / configuration split as hardening:

Family	Tests
Design vulnerabilities	**Right modes used in symmetric encryption · key size not too small** (as of 2015, **RSA under 2048 bits is insecure**) · **certificate validity** and **alerts on self-signed certificates** (**SSL-trip** can be used to avoid man-in-the-middle) · **replay attacks** (e.g. against **WEP**) · **attacks on cryptographic protocols** to verify strength [Bittau]

Family	Tests
Construction vulnerabilities	**Code reviews** — e.g. verify the **default random() is not used** to seed (relatively easy to crack) · **fuzzing** · **timing attacks** (time taken by crypto algorithms) · **power analysis** (encrypted hardware devices)
Configuration vulnerabilities	**Protocol configuration assessment** (TLS server-side, client-side authorized protocols, per a TLS configuration guide) · **TLS cipher order on the server** — can the cipher be **downgraded or renegotiated**?
Ageing	Verify mechanisms that **may have become weak and prone to being cracked**

How the official exam used this objective (K3, 2 points)

Sample Q27: what to implement to gain confidence in encryption? **Develop tests that ensure symmetric encryption is used in the right modes** — per the syllabus. Rejected: "keys **at least 256 bits**"; "**apply random algorithms** to generate random numbers" (**random() is easy to crack**); "**remove all WEP protocols** to see how the system performs" (the rationale says WEP should be **left in place**, not removed). Note: the official rationale for (a) says "**a minimum of 768 bits should be used**", which does not match the syllabus's **2048-bit RSA** statement — answer from the syllabus.

AS-5.4.1 (K2) — Understanding Firewalls and Network Zones

Original syllabus statement

Understand the concept of firewalls and the use of network zones and how they are applied in securing information systems.

"A firewall is a component or a set of components that restricts access between a protected network and the Internet, or between other sets of networks" [Chapman 2000]. It **implements and enforces a security policy** based on **authorized and forbidden communications**. It may be **host-based** (software on one host monitoring application inputs/outputs) or **network-based** (software monitoring traffic between networks). Its main task: **control traffic between trusted zones by filtering data**, so **malicious traffic from an untrusted zone is detected and blocked**.

Network zones — a sub-network with a defined level of trust

Zone	Trust
Internet / public zone	**Untrusted**
Demilitarized zones (DMZs) — several possible	**Different levels of trust**
Private / internal networks	**Most trusted**

Zones are part of the firewall configuration: they define **the authorized flows between networks**; **all forbidden traffic is blocked**.

Firewall filters on…	A WAF also filters on…
Source and target addresses and protocols (Ethernet/IP addresses, TCP/UDP ports)	**Users being the connections**
Protocol options (fragmentation, TTL)	**Data filtering** (e.g. **pattern descriptions**)
Size of data	

How the official exam used this objective (K2, 1 point)

Sample Q28: relationship between a firewall and a network zone? **A sub-network can be considered a network zone and a firewall can be traffic monitoring software.** Rejected: both focus on **size of data** (zones don't); zones **communicate through safe protocols** (zones are **part of the firewall configuration** defining authorized flows); **the zone blocks** traffic (**the firewall blocks**, not the zone).

AS-5.4.2 (K3) — Testing Firewall Effectiveness

Original syllabus statement

Demonstrate how to test the effectiveness of existing firewall implementations and network zones.

Because of **the number of protocols, their options and network complexity**, **it is difficult to configure a firewall efficiently**. Tests should include:

Test	Purpose
Port scanning	Verify **the security policy is well implemented**
Malformed network packets and network fuzzing	Exploit **unexpected behavior** — e.g. a **denial of service**
Fragmentation attacks	**Bypass filtering** to carry the attack **behind the firewall**
WAF: encode, compress or obfuscate data	**Hide the malicious information** conveying the attack

*Chapter 8's official Q42 (3 points) is a firewall scenario: after configuration checks and port scanning, the test that detects unexpected behaviour exploitable by DoS is **malformed packets or fuzzed data**.*

AS-5.5.1 (K2) — Understanding Intrusion Detection Tools

Original syllabus statement

Understand the concept of intrusion detection tools and how they are applied in securing information systems.

Attacks increase every year, techniques evolve quickly and no system is 100% safe. An **IDS** is **a standalone appliance or application** that **monitors activities at different levels — network to application, the 7 layers of the OSI model — to detect violations of the security policy**. Deviations from normal behavior raise **alerts** for further action (**traffic blocking, virtual patching**).

Two security models (IETF Intrusion Detection Exchange Format working group)

	Negative security model	Positive security model
Also called	**Signature-based / blacklist** detection	**Behavior-based / white list** detection
Rule	**"All that is not explicitly forbidden is allowed"**	**"All that is not explicitly allowed is rejected"**
Detects by	A **list of known attacks or patterns**	**Specification of expected behavior** — e.g. an input's characteristics as a **regular expression**; **trusted traffic** may generate the specification
Weakness	**Easy to bypass — only known attacks are detected**	**Large numbers of false positives and false negatives**; behaviour can be **deviated** so intrusive traffic becomes "normal"

Firewall vs IDS

A firewall looks outwardly at the traffic to stop intrusions; an IDS analyzes suspicious intrusions and raises an alert if they are confirmed.

AS-5.5.2 (K3) — Testing Intrusion Detection Tools

Original syllabus statement

Demonstrate how to test the effectiveness of existing intrusion detection tool implementations.

Target	Tests
Signature/scenario-based detection	**Evasion techniques: character encoding or modification** (white space, end of lines) · **IP fragmentation, TCP segmentation · encryption, obfuscation · URL encoding**
Behavior-based detection	Check **false negatives** (alerts that should have been reported but weren't — e.g. **new attacks**, or **rules that catch some attacks and miss others**) and **accuracy**. Because an attacker can **deviate the learned behavior** until intrusive traffic is "normal", **complementary tests should use malicious traffic in order to add new intrusive specifications considered as authorized traffic**
Test inputs	**Protection Profile for IDS** [PP-IDS] · **Web Application Firewall Evaluation Criteria** [WAFEC]

How the official exam used this objective (K3, 2 points)

Sample Q29: most effective test of an IDS? **Use tests that generate malicious traffic to add new intrusive specifications** — they target specifications **formerly considered authorized traffic**. Scenarios from past experience and known malicious traffic **might be useful but are less effective for the future**; combining IDS products is **true for usage, not for testing**.

AS-5.6.1 (K2) — Understanding Malware Scanning Tools

Original syllabus statement

Understand the concept of malware scanning tools and how they are applied in securing information systems.

- Malicious code reaches servers and end-user machines via **email attachments, fake URLs, client-side code execution**, etc.
- **Anti-malware** analyzes, detects and removes malicious code; detection targets: **malware, phishing and pharming**.
- **Main detection = signature-based**: search a database for **known patterns**. **New malware, or malware whose signature is not in the database, will not be detected.**
- A **heuristic mechanism** is often embedded to identify **slight variations of known patterns**.

How the official exam used this objective (K2, 1 point)

Sample Q30: main disadvantage of malware scanning tools? **They can only detect malware that is known to the tool.** Rejected: "only certain levels" (depends on focus, not the main disadvantage), "overly complex to run" (generally **easy to run**), "no updating or reporting" (they **do** update and report). (The official rationale mislabels the first rejection as "B"; the reasoning refers to option a.)

AS-5.6.2 (K3) — Testing Malware Scanning Tools

Original syllabus statement

Demonstrate how to test the effectiveness of existing malware scanning tool implementations.

How malware evades analysis and detection

Technique	Example
Exploiting system library functions	**FindWindow** used to **close an anti-malware application**
String obfuscation	Encryption; **JavaScript stored in a PDF**; compression with **UPX** (Ultimate Packer for Executables)
Dynamic loading of functions and libraries	Limits analysis of the malicious code
Automatic update of applications	**Skype Trojan**
Using other hardware	**GPU** decompresses code into memory — **cannot be analyzed before execution**

- Functional test: **EICAR anti-malware test file** checks effectiveness **without developing real malicious code**.
- **Test a new or upgraded anti-malware implementation on a representative platform before deploying it organization-wide** — products have **falsely identified legitimate OS files as malware and quarantined them**, **shutting down the organization's entire computing capability**.

AS-5.7.1 (K2) — Understanding Data Obfuscation

> **Original syllabus statement**
> Understand the concept of data obfuscation tools and how they are applied in securing information systems.

Obfuscation (sometimes called data masking) makes data and source code non-understandable for a human. It protects sensitive data against **copying (to bypass licence protection)** and **reverse-engineering (to understand code and exploit vulnerabilities)**.

- Lets **support personnel, functional testers, etc.** work with **non-sensitive data** while **sensitive data is hidden** — also called **data anonymization** (keeps personal data anonymous).
- Protects **source code** against **copy-paste** (e.g. an innovative algorithm) and reuse after reverse-engineering.
- Optimizing code (e.g. **parts in assembly**) can incidentally obfuscate it.
- Protects **sensitive HTML pages** (connection, administration) against **script injection**, whose success depends on knowing the site's structure.
- **Techniques: base64-encoding · XORing · randomly renaming functions · methods overriding · tab-return-space deletion · shuffling. Encryption** is also obfuscation, **but cyphered data remains viewable to those with valid keys**.
- **Attackers also use obfuscation** to hide malicious code and attacks.

AS-5.7.2 (K3) — Testing Data Obfuscation Approaches

> **Original syllabus statement**
> Demonstrate how to test the effectiveness of data obfuscation approaches.

- **Tight configuration control between obfuscated data and keys** — otherwise data **cannot be un-obfuscated**.
- Obfuscation renders **production data used in a system test environment anonymous** — e.g. a **health information system** must not divulge patient data to testers.
- **Data tests: brute force or dictionary attacks** to try to get plain data from obfuscated data.
- **Code tests: reverse-engineer Java byte code** (e.g. **Java Decompiler**) or **.NET** (e.g. **.NET Reflector**); **brute force** on vulnerable mechanisms (e.g. **unXOR**).
- **In theory code cannot protect itself against de-obfuscation, because debugging can always be used**; anti-decompilation tools exist but **risks and limitations remain**.

> **How the official exam used this objective (K3, 2 points)**

Sample Q31: removing personal identification numbers from a legacy system for testing — how to verify masking effectiveness? **Design a brute force attack on the obfuscated data.** Rejected: **manual inspection in the database** (not feasible for the volume of data and time); **substituting random data of varying lengths** (an anonymizing exercise, and field lengths may corrupt data); **stressing the database** (not the goal).

AS-5.8.1 (K2) — The Importance of Security Training

Original syllabus statement

Understand the concept of security training as a software lifecycle activity and why it is needed in securing information systems.

Humans are often the weakest link in the overall security picture. Hence **consistent and continuous training** to remind people **why policies matter** and to follow them. Training should occur **throughout the software lifecycle**, be **updated as new policies and threats arise**, and cover **identification of social engineering attacks and insider threats**.

How the official exam used this objective (K2, 1 point)

Sample Q32: weakest link in software security? **The behavior of humans.** The other options (no training plan, effort to maintain documents, advancing attack techniques) are concerns, **but people and their behavior are the weakest link**.

AS-5.8.2 (K3) — Testing the Effectiveness of Security Training

Original syllabus statement

Demonstrate how to test the effectiveness of security training.

Training topic	Effectiveness test	What a failure means
Strong, confidential passwords	**Social engineering** call from a **fake technical support person · look around desks for sticky notes** (especially **under keyboards**) · run **password audit tools** — risk: **passwords may be viewable to the person running the test**	Users need retraining
Secure coding	Tester **injects SQL** through a field with **no field-level edit** and sees the customer database	**This developer needs training**; **check other developers' code** — if widespread, a **general process improvement initiative** is needed
Physical security / clean desk	Tester attempts **unauthorized physical access** and **views documents left in the open**	Awareness and physical controls need reinforcement

Section 2 — Key Concepts

Section 1 gives the syllabus wording mechanism by mechanism. This section explains what each mechanism does, why it can fail, and how a tester proves whether it works. Each card covers one mechanism's K2 "understand" objective and its K3 "test the effectiveness" objective together, because the exam's K3 answer is always the test that exploits the mechanism's weakness.

2.1 The storyline of Chapter 5

Chapter 5 describes **eight layers of defense**. **None is enough on its own**, and each has its own strengths and weaknesses. For every mechanism, learn three things in one breath: **what it does → where it is weak → the test that attacks that weakness**. That chain is the whole chapter.

A pattern repeats across hardening and encryption: vulnerabilities come **by design** (a wrong architecture or choice), **by construction** (a coding defect) or **by configuration** (a bad setting). When a question asks what kind of test to run, first decide which of the three the weakness is.

2.2 The eight mechanisms

1 System hardening (AS-5.1.1 / 5.1.2)	
In plain words	Every extra program, open port, default account or old library is another **door an attacker can try**. **Hardening** is the **step-by-step removal and locking of those doors** (reducing the **attack surface**) by applying the security policy and several layers of protection.
How it works	**Remove** what is not needed: software, libraries and **developer tools**, accounts, applications and **network services**, peripherals and **hardware ports** (USB, card readers). **Patch** promptly (e.g. automatic updates), **update configurations**, and **follow coding rules** (to avoid by-construction flaws). **Send logs to a remote log server** (e.g. rsyslog). An attacker who takes over the machine can delete the local logs, but **not** the copies on the remote server. Hardening uses the other mechanisms too: strong authentication, least-privilege authorization, encryption, firewalls and **zones/sandboxes**, IDS, anti-malware and obfuscation. **Balance it with usability.** At the extreme, protections get **switched off** because they stop people working.
How to test it	**Tests: audit configurations** (are **default passwords changed**? which **services and ports** are open?) · check that **component, library and application versions are not obsolete** · run a **vulnerability scanner** (especially for complex, multi-site systems) · use **static analysis** to find coding-rule violations. **Also test for over-hardening**: protections that are **too restrictive** for the actual risk.
Example	A new web server ships with a sample application, the default admin password, and FTP enabled. Hardening removes the sample app, changes the password, disables FTP, applies patches and forwards logs. The test is a configuration audit plus a vulnerability scan that confirms only ports 443 and 22 respond and that no default credentials work.
Exam angle	Official Q25: how to ensure hardening works as expected → **closely monitor security performance reports and metrics to determine whether the correct level of access and authentication is achieved.** Reject auditing **only** strong authentication (just one aspect), seeking **equal** hardening everywhere (critical areas deserve more), and **hiring a known hacker** (they may not tell you what they found).
Don't confuse	Hardening is a **process applied by policy**, not a single product. A firewall is **one** of the mechanisms hardening uses.

2 Authentication and authorization (AS-5.2.1 / 5.2.2)	
In plain words	**Authentication** answers **"who are you, and are you really that person?"** (the login and the password, token, certificate or fingerprint). **Authorization** answers **"now that we know who you are, what are you allowed to do?"** Authentication always comes **first**. A user who is not authenticated has **no rights, or only restricted ones**.

2 Authentication and authorization (AS-5.2.1 / 5.2.2)	
How it works	**Authentication** checks a **user identifier plus a token**. Implementations: detecting **weak passwords**, **one-time passwords**, **fingerprints**, **software certificates**, **certificates in hardware tokens**. Architectures: **local, server, network, single sign-on**. **Authorization** grants a **level of access**. For example, you can log into a server but **not modify its data**, or you can use FTP **only in your own directory**. **Attackers exploit:** coding errors (**no input filtering**), **old vulnerable libraries**, configuration errors (**default passwords and rights**), and **weak passwords** ("123456" is the most common).
How to test it	**Tests: brute force and dictionary** attacks (123456, 111111, birth dates, pets' names) · **SQL injection** to log in **without** valid credentials · **unauthorized URLs/URIs** ("../../" out of an FTP directory, "/admin" on a website) · **exploiting an unpatched vulnerability** to gain control and **escalate privileges**. Password rules must follow **current good practice** (e.g. SANS guidelines), and they only help if users follow them.
Example	A shop: an anonymous visitor can **browse** (no authentication needed). To **buy**, they create an account and log in (authentication). Once logged in they can purchase, but they **cannot reach the admin pages** (authorization). A tester types "/admin" while logged in as a customer. If the page opens, **authorization** has failed even though authentication worked.
Exam angle	Official Q26: the key attribute of **authentication → it verifies that the user entering the system is legitimate.** "Correct profile and rights to parts of the system" describes **authorization**. Everyone should have **unique** credentials, so shared credentials are wrong.
Don't confuse	**AuthN = identity. AuthZ = permissions.** SQL injection on the login page attacks **authentication**. "/admin" as a normal user attacks **authorization**.

3 Encryption (AS-5.3.1 / 5.3.2)	
In plain words	Encryption turns readable data (**plain text**) into unreadable data (**cypher text**) using an **algorithm and a secret key**, so that only people who hold the secret can turn it back. The goal is that a thief who **steals the encrypted data still cannot read it**.
How it works	It protects data **at rest** (encrypted passwords in a database, an encrypted drive) and **in transit** (encrypted email, **SSL/TLS**). **Symmetric** encryption: **one shared secret key** encrypts and decrypts. **Asymmetric** encryption: a **public key** and a **private key**. **Hashing and salting** are methods used alongside encryption (for example, for stored passwords). Cryptography supports **confidentiality, integrity, availability and non-repudiation**.
How to test it	**Design tests:** symmetric encryption used in the **right modes · key size not too small** (**RSA under 2048 bits is insecure, as of 2015**) · **certificates valid**, and **alerts raised on self-signed certificates** (SSL-trip-style checks against man in the middle) · **replay attacks** (e.g. against WEP) · attacks on the protocol itself. **Construction tests: code review** (e.g. the **default random() function must not be used** for seeding, because it is easy to predict) · **fuzzing · timing attacks · power analysis** on encryption hardware. **Configuration tests:** assess the **TLS configuration** on server and client, and check the server's **cipher order**: can a connection be **downgraded or renegotiated** to a weak cipher? **Ageing:** check for mechanisms that have **become weak** over time.
Example	A banking app uses AES (good) in a mode that encrypts identical blocks to identical output (a **wrong mode** → design test). Its session keys are seeded with the default random() (**construction** → code review). Its server still accepts an old TLS version first in the cipher list (**configuration** → TLS assessment).
Exam angle	Official Q27: to gain confidence in encryption → **develop tests that ensure symmetric encryption is used in the right modes.** Reject "keys at least 256 bits" (not the syllabus rule), "apply random algorithms" (random() is easy to crack), and "remove all WEP" to see what happens. For key sizes, answer from the **syllabus (RSA ≥ 2048)**, not from the sample-exam rationale's "768 bits".

3 Encryption (AS-5.3.1 / 5.3.2)	
Don't confuse	**Encryption is reversible with the key. Hashing is one-way.** Obfuscation (mechanism 7) also hides data, but encrypted data is still readable by anyone with a **valid key**.

4 Firewalls and network zones (AS-5.4.1 / 5.4.2)	
In plain words	A **network zone** is a **part of the network with a defined level of trust**: the Internet is **untrusted**, a **DMZ** is **in between**, and the internal network is **most trusted**. A **firewall** is the **guard between zones**. It applies the security policy by **allowing the permitted traffic and blocking everything else**.
How it works	**Definition (Chapman):** a component or set of components that **restricts access between a protected network and the Internet, or between other sets of networks**. **Host-based** firewall: software on **one machine**, watching that machine's applications. **Network-based** firewall: watches traffic **between networks**. It filters on **source and destination addresses and protocols** (IP addresses, TCP/UDP ports), **protocol options** (fragmentation, TTL) and **data size**. A **web application firewall (WAF)** also filters on **the users making the connections** and on **data patterns**. **Zones are part of the firewall configuration.** They define which flows between networks are allowed.
How to test it	Firewalls are **hard to configure well** because of the number of protocols and options, and because networks are complex. **Tests: port scanning** (is the policy really implemented?) · **malformed packets and network fuzzing** (unexpected behaviour, e.g. **DoS**) · **fragmentation attacks** (split the attack so the filter misses it and it is **reassembled behind the firewall**) · for a WAF, **encode, compress or obfuscate** the payload to hide it.
Example	Policy: only HTTPS from the Internet may reach the DMZ web server, and only the web server may reach the internal database. A port scan from outside should show **only 443** open on the web server and **nothing** on the database. Then send malformed TCP packets to check that the firewall rejects them instead of crashing.
Exam angle	Official Q28: the relationship → **a sub-network can be considered a network zone, and a firewall can be traffic-monitoring software.** The **firewall** blocks traffic, not the zone. Zones do not filter on data size. In Chapter 8's Q42 (3 points), the DoS worry after port scanning → **malformed packets or fuzzed data**.
Don't confuse	**Firewall vs IDS:** a firewall **looks at the traffic to stop** intrusions. An IDS **analyzes** suspicious activity and **raises an alert**.

5 Intrusion detection systems (AS-5.5.1 / 5.5.2)	
In plain words	No system is 100% safe, so you also need to **notice attacks**. An **IDS** watches activity at **all levels, from network to application (the 7 OSI layers)**, spots **violations of the security policy**, and raises **alerts** for action (blocking the traffic, virtual patching).
How it works	**Negative security model** (signature-based, **blacklist**): "**everything not explicitly forbidden is allowed**". It looks for **known attack patterns**. **Weakness: easy to bypass, because only known attacks are detected**. **Positive security model** (behaviour-based, **white list**): "**everything not explicitly allowed is rejected**". It compares activity with a **specification of expected behaviour**, which can be learned from trusted traffic. **Weakness:** many **false positives and false negatives**, and an attacker can slowly **shift what counts as normal**.
How to test it	**Tests of signature detection** use **evasion**: **character encoding or modification** (white space, line endings), **IP fragmentation and TCP segmentation**, **encryption and obfuscation**, **URL encoding**. **Tests of behaviour detection** check **false negatives** (alerts that should have fired but did not, e.g. on **new** attacks) and **accuracy**. Because the learned behaviour can drift, **send malicious traffic to add new intrusive specifications that had been considered authorized traffic**. Inputs for test design: the **Protection Profile for IDS** and the **WAF Evaluation Criteria**.

5 Intrusion detection systems (AS-5.5.1 / 5.5.2)	
Example	A signature IDS catches "UNION SELECT". The tester sends "UN%49ON SEL%45CT" (URL-encoded) and the IDS stays silent: an **evasion** gap. A behaviour IDS has learned that the admin page gets 10 requests a day. The tester slowly raises this to 1,000 over a month and checks whether the IDS ever objects: **drift**.
Exam angle	Official Q29: the most effective IDS test → **generate malicious traffic to add new intrusive specifications** (traffic that was formerly treated as authorized). Replaying past scenarios or known malicious traffic is **less effective for the future**.
Don't confuse	**False positive** = an alarm on harmless activity. **False negative** = **no alarm on a real attack** (the syllabus: "an alert that should have been reported but wasn't"). The negative and positive models are the **IDS rule philosophies**, not test results.

6 Malware scanning (AS-5.6.1 / 5.6.2)	
In plain words	Anti-malware **finds and removes malicious code** (and detects phishing and pharming). Its main method is **signature matching**: it compares files with a database of **known** malware. **Its built-in weakness: it can only detect malware it already knows.** Heuristics help it catch **slight variations** of known patterns.
How it works	Malware arrives through **email attachments, fake URLs and client-side code execution**. **How malware evades scanners:** using **system library functions** (FindWindow to **close the anti-malware program**) · **string obfuscation** (encryption, **JavaScript hidden in a PDF**, **UPX** packing) · **dynamic loading** of functions · **automatic updates** of applications (the **Skype Trojan**) · using **other hardware** (the **GPU** unpacks code into memory, so it **cannot be analyzed before it runs**).
How to test it	**Functional test:** the **EICAR test file**, a harmless file every scanner treats as malware, so you can test detection **without writing real malicious code**. **Before deploying a new or upgraded anti-malware product organization-wide, test it on a representative platform.** Products have **quarantined legitimate OS files** by mistake and **shut down an organization's entire computing capability**.
Example	IT plans to push a new antivirus version to 8,000 PCs on Friday. Tested first on a pilot group with the standard desktop image, it flags a core Windows DLL as malicious. The pilot prevents a company-wide outage.
Exam angle	Official Q30: the main disadvantage → **it can only detect malware that is known to the tool.** Reject "overly complex to run" (it is easy to run) and "no updating or reporting" (it does update and report).
Don't confuse	**Anti-malware vs IDS:** both are often signature-based and share the "only knows the known" weakness, but anti-malware looks at **code and files**, while an IDS looks at **activity and traffic**.

7 Data obfuscation (AS-5.7.1 / 5.7.2)	
In plain words	**Obfuscation** (also called **data masking**) makes data or source code **unintelligible to a person**, while it can still be used. It protects against **copying** (to bypass licence protection) and **reverse-engineering** (to find vulnerabilities).
How it works	**Uses:** let support staff and functional testers work with **non-sensitive** copies of data (**anonymization**) · protect an innovative **algorithm** in source code · make **sensitive HTML pages** (login, admin) harder to attack with script injection, which depends on knowing the page structure. **Techniques: base64 encoding, XORing, randomly renaming functions, method overriding, removing tabs, returns and spaces, shuffling**. Encryption is also a kind of obfuscation, but encrypted data **remains readable to anyone with a valid key**. **Attackers obfuscate too**, to hide malicious code. **Configuration control** between obfuscated data and its **keys** must be tight, or the data **cannot be un-obfuscated** when needed.

7 Data obfuscation (AS-5.7.1 / 5.7.2)	
How to test it	**Data tests: brute force or dictionary attacks** to recover the original data from the masked data. **Code tests: decompile** Java byte code (**Java Decompiler**) or .NET (**.NET Reflector**), and **brute force** weak schemes (**unXOR**). **In theory, code cannot fully protect itself against de-obfuscation, because a debugger can always be used.** Anti-decompilation tools reduce the risk but do not remove it.
Example	A hospital copies production patient data into the test environment with names and ID numbers masked. The tester runs a dictionary attack: masked ID numbers that keep the same check digit and date-of-birth prefix can be matched back to real patients in minutes. The masking **failed**.
Exam angle	Official Q31: to verify that masking of personal ID numbers worked → **design a brute-force attack on the obfuscated data.** Manual inspection of the database is **not feasible** at that volume. Substituting random data is **anonymizing, not testing**. Stressing the database is **not the goal**.
Don't confuse	**Obfuscation hides meaning. Encryption needs a key and can be reversed with it.** In testing, obfuscation's main job is **anonymizing production data**, so testers never see real personal data.

8 Security training (AS-5.8.1 / 5.8.2)	
In plain words	**Humans are often the weakest link** in security. Training must be **consistent and continuous**, run **throughout the software lifecycle**, be **updated** when new policies and threats appear, and cover how to **recognise social engineering and insider threats**. You test training by **checking whether people actually behave as they were taught**.
How it works	**Passwords training → tests:** a **social-engineering call** from a fake technical-support person asking for the password · **looking for sticky notes** on desks and **under keyboards** · running **password audit tools** (with care: the tester may **see** the passwords). **Secure-coding training → test:** a tester **injects SQL** through a field with **no field-level validation** and reaches the customer database. **That developer needs training.** Check other developers' code: if the problem is widespread, a **general process-improvement** initiative is needed. **Physical security / clean-desk training → test:** attempt **unauthorized physical access** and look for **documents left in the open**.
Example	After a phishing-awareness course, the security team sends a realistic fake "parcel delivery" email. 18% of staff click, compared with 31% before the course. Training helped, but not enough, so the next round targets the departments with the worst results, using this test as a real case study (Chapter 6).
Exam angle	Official Q32: the weakest link → **the behavior of humans.** Missing training plans, document maintenance and new attack techniques are concerns, but **people** are the weakest link.
Don't confuse	Testing training is **testing behaviour**, not marking an exam paper. A course completion rate is **not** evidence of effectiveness.

2.3 Fuzzy points clarified

Places where the wording is easy to misread

Key size: syllabus vs sample-exam rationale. The syllabus says **RSA under 2048 bits is insecure (as of 2015)**. The official Q27 rationale mentions "a minimum of 768 bits". Answer key-size questions **from the syllabus**. The correct option in Q27 (right modes) does not depend on this.

"Repudiation" in §5.3.1 means **non-repudiation** (being unable to deny an action).

"Negative" and "positive" models are not about results. Negative = blacklist of known bad. **Positive** = whitelist of known good. Do not link them to false negatives and false positives just because of the word.

Hashing and salting are "methods used during encryption" in the syllabus, even though hashing on its own is one-way and is not encryption. Accept the syllabus framing for the exam.

Q25's correct answer (monitor security performance reports and metrics) is not in the syllabus's list of hardening tests. It is the best of the four options. The syllabus's own tests are configuration audits, version checks, vulnerability scanners and static

analysis.

SSL-trip: the syllabus names it as a way to check certificates against man in the middle. You only need to recognise the name, not know how to use it.

2.4 Chapter 5 keyword glossary

Keyword	Working definition for the exam
anti-malware	Software to analyze, detect and remove malicious code; mainly signature-based plus heuristics.
authentication	Verifying a user identifier and token — who the user is and whether they are who they claim.
authorization	Verifying an authenticated user's rights to an action and level of access.
demilitarized zone	A network zone between untrusted and internal networks with an intermediate level of trust.
encryption	Encoding plain text into cypher text with an algorithm and secrets so only authorized persons can decrypt.
firewall	Component(s) restricting access between a protected network and the Internet or other networks.
hashing	A one-way transformation used during encryption (e.g. of stored passwords).
insider threat	A threat from an authorized user inside the organization.
intrusion detection system	Appliance or application monitoring activity to detect security-policy violations and raise alerts.
malware	Malicious code placed at a target via email, fake URLs, client-side execution, etc.
malware scanning	Detecting malware by known signatures and heuristics.
network zone	A sub-network with a defined level of trust.
pharming	Redirecting users to fraudulent sites; a detection target of anti-malware.
phishing	Luring users to reveal information or visit infected sites; a detection target of anti-malware.
salting	Adding random data before hashing, used during encryption to strengthen stored secrets.
system hardening	Step-by-step reduction of the attack surface by applying a security policy and layers of protection.
vulnerability scanner	A tool that eases vulnerability assessment, especially in complex systems.

Section 3 — Tables and Summaries

3.1 Mechanism → weakness → test (the K3 decider)

Mechanism	Weakness the syllabus names	Test the syllabus names
Hardening	Defaults, unnecessary services and ports, obsolete versions	Configuration audit; vulnerability scanner; static analysis
AuthN/AuthZ	Weak passwords, no input filtering, default rights, old libraries	Brute force/dictionary; SQLi; URI/URL tampering; exploit unpatched flaw
Encryption	Short/static keys, wrong modes, bad randomness, poor configuration, ageing	Key size; modes; certificates; code review; timing/power; TLS config; ageing
Firewall	Hard to configure efficiently	Port scan; malformed packets/fuzzing; fragmentation; WAF encoding
IDS	Signatures miss unknown attacks; behaviour models drift	Evasion encodings; malicious traffic to add intrusive specifications
Anti-malware	Only detects known malware; false positives can kill systems	EICAR; representative platform first
Obfuscation	Reversible; keys must match	Brute force/dictionary; decompilers; unXOR
Training	Humans	Social engineering call; desk sweep; password audit; SQLi; physical access

3.2 Design / construction / configuration — the recurring triad

Vulnerability type	Hardening example	Encryption example
By design	Wrong architecture	Wrong mode; small key; self-signed certificate; replayable protocol
By construction	Coding-rule violation (static analysis)	random() seed; fuzzing; timing; power analysis
By configuration	Default passwords; open ports	TLS configuration; cipher order downgrade

3.3 Negative vs positive security models

	Negative	Positive
Rule	Not forbidden → allowed	Not allowed → rejected
Also called	Signature / blacklist	Behaviour / white list
Fails by	Missing unknown attacks (bypass)	False positives and negatives; drift
Test with	Evasion encodings	Malicious traffic to add intrusive specifications

Reproduce from memory before moving on

Table 3.1 — every K3 question in this chapter is a row of it.

Authentication vs authorization and **firewall vs zone vs IDS** — the usual K2s.

Humans are the weakest link — the training K2.

Section 4 — Demo Questions with Answers

*Questions 1–8 are the **official ISTQB CTAL-SEC Sample Exam questions for Chapter 5** — #25 to #32 — with official answers and rationale. LO tags are inferred and match the Exam Structure Table. Questions 9–32 are derived strictly from the syllabus, three per mechanism. Correct options are marked ✔.*

Part A — The 8 official Chapter 5 questions

Q1. OFFICIAL · Sample Exam #25 (2 pts)

You are implementing procedures for evaluating system hardening in an effort to test the system's security effectiveness. What procedure might you follow to ensure the hardening mechanisms put in place are working as expected?

a) Closely monitor various security performance reports and metrics to determine if the correct level of access and authentication is achieved ✔

b) Frequently audit strong authentication to ensure a high level of intrusion protection is maintained at all times

c) Evaluate the hardware components that have been hardened and compare these to other hardened software components to ensure equilibrium is being achieved

d) Enlist a known hacker to conduct an independent assessment of the hardening effectiveness

Correct answer: a | LO: AS-5.1.2 (K3) | Syllabus §5.1.2

Why: (a) — official rationale: **there are security performance reports and metrics available that can be used to determine if you have achieved the right level of hardening.** (b) **strong authentication is just one aspect of hardening**. (c) **equilibrium is not needed. The more critical areas may warrant better hardening.** (d) **there is the danger of the hacker not telling you what is found.** Syllabus §5.1.2: tests should show hardening is **working, applied in the right places and in the right ways**.

Q2. OFFICIAL · Sample Exam #26 (1 pt)

What are key attributes of security authentication of a medium complexity IT system?

a) It verifies that the user has the correct profile and corresponding rights to access limited parts of the system

b) It is key in identifying the amount of system resources the user can utilize

c) It verifies that user entering the system is legitimate ✔

d) It uses common credentials among users to gain entry into the system

Correct answer: c | LO: AS-5.2.1 (K2) | Syllabus §5.2.1

Why: (c) — official rationale: **it verifies that the user is legitimate and authorized**. (a) **is not looking at access rights** (that is authorization). (b) **system resource utilization is not a consideration**. (d) **common credential verification should not be used – each individual should have unique credentials**.

Q3. OFFICIAL · Sample Exam #27 (2 pts)

Typical encryption mechanisms are vulnerable to threats which makes it important to understand their effectiveness at any given time. Identify which of the following you should implement to gain confidence in your encryption mechanisms?

a) Evaluate cryptographic keys to ensure they are at least 256 bits in size

b) Ensure you are applying random algorithms to generate random numbers where possible

c) Develop tests that ensure symmetric encryption is used in the right modes ✔

d) Remove all WEP protocols to see how the system performs

Correct answer: c | LO: AS-5.3.2 (K3) | Syllabus §5.3.2

Why: (c) — official rationale: **correct per the syllabus**. (b) **the random algorithm is easy to crack**. (d) **WEP protocols should be left in place, not removed**. For (a) the official rationale says **"a minimum of 768 bits should be used"**; the syllabus itself says **as of 2015 an RSA key less than 2048 bit is considered insecure**. Either way (a) is not the syllabus test — **assessment that right modes are used in symmetric encryption** is.

Q4. OFFICIAL · Sample Exam #28 (1 pt)

Which is true of the relationship between a firewall and a network zone?

a) Both a network zone and firewall focus on the size of data that is being passed through

b) A network zone communicates through safe protocol options while a firewall ensures those protocols are safe

c) A sub-network can be considered a network zone and a firewall can be traffic monitoring software ✔

d) A network zone blocks malicious traffic from an untrusted zone which the firewall does not filter

Correct answer: c | LO: AS-5.4.1 (K2) | Syllabus §5.4.1

Why: (c) — official rationale: **correct per the syllabus**. (a) **network zones do not focus on size of data**. (b) **network zones are parts of the configuration of the firewall and define the authorized flow of data between networks**. (d) **the firewall blocks the traffic, not the network zone**. Syllabus: **a network zone is an identified sub-network with a defined level of trust**; a network-based firewall is **software that monitors traffic between networks**.

Q5. OFFICIAL · Sample Exam #29 (2 pts)

Which of the following would you apply to most effectively test the abilities of an intrusion detection tool?

a) Develop a series of scenarios based on past experience

b) Use tests that generate malicious traffic to add new intrusive specifications ✔

c) Apply it to situations of known malicious traffic

d) Use it in conjunction with other IDS products when possible

Correct answer: b | LO: AS-5.5.2 (K3) | Syllabus §5.5.2

Why: (b) — official rationale: **these tests can be used to add new intrusive specifications which were formerly considered to be authorized traffic.** (a) and (c) **might be useful, but will not be as effective as B in making sure the tool will work for the future as well as the present**. (d) **is true for the usage, but not for the testing**.

Q6. OFFICIAL · Sample Exam #30 (1 pt)

Which of the following is a main disadvantage to malware scanning tools?

a) They only detect certain levels of malware

b) They can only detect malware that is known to the tool ✔

c) They tend to be overly complex to run

d) They do not provide updating and reporting capabilities

Correct answer: b | LO: AS-5.6.1 (K2) | Syllabus §5.6.1

Why: (b) — official rationale: **the malware tool can only detect malware that it already knows about.** (a) **may be correct depending on the particular focus of the tool, but is not a main disadvantage** (the official text labels this "B"; it refers to option a). (c) **the tools are normally easy to run**. (d) **the tools provide the ability to update themselves with new findings and to produce reports**.

Q7. OFFICIAL · Sample Exam #31 (2 pts)

You need to remove personal identification numbers from a legacy system in order to reduce risk during testing. Part of your data obfuscation plan includes verifying how effectively the data is masked. Which of the following is the most effective approach to use?

a) Manually verify in the database that the data targeted for obfuscation is no longer understandable for logical human interpretation

b) Design a brute force attack on the obfuscated data ✔

c) Substitute the sensitive data with random data of varying string lengths

d) Enlist the development teams to generate a program to stress the database for vulnerabilities

Correct answer: b | LO: AS-5.7.2 (K3) | Syllabus §5.7.2

Why: (b) — official rationale: **a brute force or dictionary attack can be used to see if personal information is still accessible.** (a) **is generally not feasible because of the amount of data and time it would take**. (c) **is more of an anonymizing exercise. Also,**

the field length might be limited so this may corrupt the data. (d) **we are not trying to stress test the database itself**.

Q8. OFFICIAL · Sample Exam #32 (1 pt)

What is often considered the weakest link in software security?

a) The lack of a consistent and comprehensive security training plan

b) The effort necessary to maintain document and procedure updates in order to keep up with continuing security threats

c) The behavior of humans ✔

d) The constant advancement in malicious techniques

Correct answer: c | LO: AS-5.8.1 (K2) | Syllabus §5.8.1

Why: (c) — official rationale: **it's the people and their behavior that is the weakest link.** Syllabus §5.8.1: **humans are often the weakest link in the overall security picture.**

Part B — 24 additional syllabus-derived questions

*The K-level in each question's label is the level of the **learning objective** it practises, not of the question itself. Some Part B items are quick recall checks on a K3 or K4 objective. Use them to learn the facts, and use the scenario items (and Part A) to practise the reasoning the real K3/K4 questions need.*

Q9. Derived from syllabus

Why does the syllabus recommend configuring a remote logging server (e.g. rsyslog) as part of system hardening?

a) To speed up log analysis

b) So that if a machine is compromised, the attacker can delete log files only on the compromised machine, not on the remote log server ✔

c) To comply with software licensing policies

d) To remove the need for an IDS

Correct answer: b | LO: AS-5.1.1 (K2) | Syllabus §5.1.1

Why: (b) — verbatim reasoning from §5.1.1.

Q10. Derived from syllabus

A default administrator password left unchanged on a database server is an example of which kind of vulnerability?

a) By design

b) By construction

c) By obsolescence

d) By configuration ✔

Correct answer: d | LO: AS-5.1.1 (K2) | Syllabus §5.1.1

Why: (d). §5.1.1: vulnerabilities come from **design errors (by design)**, **source code defects (by construction)** or **lack of rigor in the configuration (by configuration)**.

Q11. Derived from syllabus

A multi-site retailer has hardened 300 store servers. Management wants evidence that hardening is effective without slowing store operations.

Which set of tests best follows the syllabus?

a) Audit server and database configurations for changed default passwords and unnecessary services and ports; verify component and library versions are not obsolete; run a vulnerability scanner across sites; and check that hardening is not so restrictive that it blocks store work ✔

b) Disable all USB ports and assume compliance

c) Ask store managers to confirm by email that servers are hardened

d) Run a load test on one store server

Correct answer: a | LO: AS-5.1.2 (K3) | Syllabus §5.1.2

Why: (a) combines the syllabus tests — configuration audits (**default passwords**, **unnecessary services and network ports**), **version verification**, a **vulnerability scanner** for **complex multi-site** environments — with the instruction to **test for protections that are too restrictive**.

Q12. Derived from syllabus

A user can log in to an FTP server but can only write to their own dedicated directory. Which mechanism enforces the directory restriction?

a) Authentication

b) Encryption

c) Authorization ✔

d) Obfuscation

Correct answer: c | LO: AS-5.2.1 (K2) | Syllabus §5.2.1

Why: (c) — the syllabus's own example of authorization: **a user is authorized to use an FTP server but only on their dedicated space**.

Q13. Derived from syllabus

Which of the following is listed by the syllabus as an authentication architecture option?

a) Single Sign-On (SSO) ✔

b) Network zones

c) Data masking

d) Virtual patching

Correct answer: a | LO: AS-5.2.1 (K2) | Syllabus §5.2.1

Why: (a). Architectures: **local authentication, server authentication, network authentication, Single Sign-On**. Implementations: weak-password detection, **OTP**, fingerprinting, software certificates, certificates in hard tokens.

Q14. Derived from syllabus

Which test exploits a lack of input filtering to bypass authentication?

a) Trying "123456" as a password

b) Entering ../../ in an FTP path

c) Checking the TLS cipher order

d) Injecting SQL into the login fields to be authenticated without any known login/password ✔

Correct answer: d | LO: AS-5.2.2 (K3) | Syllabus §5.2.2

Why: (d) — §5.2.2. (a) is **brute force/dictionary**; (b) is an **unauthorized URI** test; (c) is an encryption configuration test.

Q15. Derived from syllabus

A web shop's admin console is at /admin and is "hidden" from the menu. Customer accounts use passwords of at least 6 characters. The shop runs a framework version two years old.

Which set of tests best evaluates authentication and authorization effectiveness?

a) Measure response time of /admin

b) Request /admin as an anonymous and as a customer user; run brute force and dictionary attacks starting with "123456", "111111", dates of birth and pet names; test login fields for SQL injection; check whether the old framework has a known exploitable vulnerability leading to privilege escalation ✔

c) Review the marketing copy on the login page

d) Confirm passwords are at least 6 characters long

Correct answer: b | LO: AS-5.2.2 (K3) | Syllabus §5.2.2

Why: (b) applies all four syllabus test types: **unauthorized URL (site address/admin)**, **brute force and dictionary**, **SQL injection**, and **exploiting an unpatched vulnerability** for **privilege escalation**. (d) only re-checks the rule, not its effectiveness.

Q16. Derived from syllabus

What distinguishes asymmetric from symmetric encryption, as described in the syllabus?

a) Asymmetric uses a private and public key; symmetric uses a shared secret key ✔

b) Asymmetric is used only for stored data; symmetric only for communication

c) Asymmetric uses hashing; symmetric uses salting

d) Symmetric encryption is always weaker

Correct answer: a | LO: AS-5.3.1 (K2) | Syllabus §5.3.1

Why: (a) — §5.3.1. Both may protect data **stored** (cyphered passwords, drives) or **in communication** (email, SSL/TLS).

Q17. Derived from syllabus

A code review finds that session tokens are generated using the language's default random() function. Which category of encryption vulnerability does this test address?

a) Design vulnerability

b) Configuration vulnerability

c) Construction vulnerability ✔

d) Ageing vulnerability

Correct answer: c | LO: AS-5.3.2 (K3) | Syllabus §5.3.2

Why: (c) — §5.3.2 lists **code reviews (e.g. to verify that the default function random() is not used to generate random numbers (seed) because the random algorithm is relatively easy to crack)** under **construction vulnerabilities**, with fuzzing, timing attacks and power analysis.

Q18. Derived from syllabus

A tester checks whether an attacker can force the server to negotiate a weaker cipher. Which syllabus test is this?

a) Power analysis

b) Replay attack against WEP

c) Timing attack

d) TLS cipher order on the server side, to see if the cipher can be downgraded or renegotiated ✔

Correct answer: d | LO: AS-5.3.2 (K3) | Syllabus §5.3.2

Why: (d) — a **configuration** vulnerability test in §5.3.2.

Q19. Derived from syllabus

Which zone is considered the most trusted?

a) The Internet / public zone

b) Private / internal networks ✔

c) The DMZ

d) All zones are equally trusted

Correct answer: b | LO: AS-5.4.1 (K2) | Syllabus §5.4.1

Why: (b) — the syllabus: Internet/public **untrusted**; DMZs with **different levels of trust**; private/internal **most trusted**.

Q20. Derived from syllabus

What can a Web Application Firewall filter on that a traditional firewall typically does not, according to the syllabus?

a) TCP/UDP ports

b) Fragmentation and TTL options

c) Users being the connections, and data using pattern descriptions ✔

d) Source and destination IP addresses

Correct answer: c | LO: AS-5.4.1 (K2) | Syllabus §5.4.1

Why: (c). Firewalls filter **addresses and protocols, protocol options, size of data**; WAFs **also** filter **users being the connections** and **data (pattern descriptions)**.

Q21. Derived from syllabus

Which test attempts to bypass firewall filtering so that an attack can be carried out behind the firewall?

a) Fragmentation attacks ✔

b) Port scanning

c) Password auditing

d) EICAR test file

Correct answer: a | LO: AS-5.4.2 (K3) | Syllabus §5.4.2

Why: (a) — §5.4.2: **fragmentation attacks to bypass filtering features with the objective to carry on the attack behind the firewall**. Port scanning verifies **the policy is well implemented**.

Q22. Derived from syllabus

Which statement describes the positive security model for intrusion detection?

a) All that is not explicitly forbidden is allowed

b) Detection is based on a list of known attacks

c) It is also called blacklist detection

d) All that is not explicitly allowed is rejected; detection is based on a specification of the expected behavior of the system ✔

Correct answer: d | LO: AS-5.5.1 (K2) | Syllabus §5.5.1

Why: (d). (a), (b) and (c) describe the **negative** (signature-based, blacklist) model.

Q23. Derived from syllabus

How does an IDS differ from a firewall?

a) An IDS blocks all traffic from untrusted zones

b) A firewall looks outwardly at the traffic to stop intrusions, whereas an IDS analyzes suspicious intrusions and raises an alert if they are confirmed ✔

c) An IDS only works at the network layer

d) A firewall cannot be software

Correct answer: b | LO: AS-5.5.1 (K2) | Syllabus §5.5.1

Why: (b) — §5.5.1. An IDS monitors **all 7 OSI layers**, so (c) is wrong; firewalls can be **host-based software** or network software, so (d) is wrong.

Q24. Derived from syllabus

A signature-based IDS is to be tested for robustness against evasion. Which set of tests fits the syllabus?

a) Port scanning and password cracking

b) Load testing at peak volume

c) Character encoding or modification (white space, end of lines), IP fragmentation and TCP segmentation, encryption and obfuscation, URL encoding ✔

d) Comparing two IDS vendors' marketing claims

Correct answer: c | LO: AS-5.5.2 (K3) | Syllabus §5.5.2

Why: (c) — §5.5.2: **scenario-based detection is easy to bypass because only known attacks are detected**; the listed **evasion techniques** test it.

Q25. Derived from syllabus

What helps anti-malware detect slight variations of known malicious patterns?

a) A heuristic mechanism ✔

b) A larger firewall rule set

c) Data obfuscation

d) Single Sign-On

Correct answer: a | LO: AS-5.6.1 (K2) | Syllabus §5.6.1

Why: (a) — §5.6.1: **a heuristic mechanism often is embedded in anti-malware to identify slight variations of known malicious patterns**.

Q26. Derived from syllabus

How can the effectiveness of an anti-malware product be functionally tested without developing real malicious code?

a) By disabling the product and observing

b) By running a password audit tool

c) By fragmenting IP packets

d) By using the EICAR anti-malware test file ✔

Correct answer: d | LO: AS-5.6.2 (K3) | Syllabus §5.6.2

Why: (d) — §5.6.2.

Q27. Derived from syllabus

An organization plans to roll out a new anti-malware version to all 8,000 desktops and servers over a weekend.

What is the most important test consideration, according to the syllabus?

a) Measure how long a full scan takes

b) Test the implementation on a representative platform before deploying to the entire organization, because anti-malware has falsely quarantined legitimate OS files and shut down organizations' computing capability ✔

c) Disable the heuristic engine to avoid false positives

d) Deploy first and monitor the helpdesk for complaints

Correct answer: b | LO: AS-5.6.2 (K3) | Syllabus §5.6.2

Why: (b) — §5.6.2's explicit warning.

Q28. Derived from syllabus

Why is encryption described as an obfuscation technique "with issues"?

a) It cannot be applied to source code

b) It is too slow for databases

c) The cyphered data remains viewable to those with valid keys ✔

d) It cannot be reversed

Correct answer: c | LO: AS-5.7.1 (K2) | Syllabus §5.7.1

Why: (c) — §5.7.1: **encryption is also an obfuscation technique but with issues because the cyphered data will remain viewable to those with valid keys**.

Q29. Derived from syllabus

To verify that a commercial Java application's code obfuscation protects a proprietary algorithm, which test fits the syllabus?

a) Attempt to reverse-engineer the byte code with a Java decompiler, and attempt brute force against weak obfuscation mechanisms (e.g. unXOR) ✔

b) Encrypt the source repository

c) Run a port scan on the application server

d) Count the lines of obfuscated code

Correct answer: a | LO: AS-5.7.2 (K3) | Syllabus §5.7.2

Why: (a) — §5.7.2 tests to verify obfuscation of code: **reverse-engineering of Java byte code (Java Decompiler) or .NET programs (.NET Reflector)** and **brute force attacks (e.g. unXOR)**.

Q30. Derived from syllabus

After obfuscating a test database, testers discover some records cannot be restored for a reconciliation test. What is the most likely cause, per the syllabus?

a) Obfuscation never allows data to be restored

b) The database was too large

c) Brute force attacks corrupted the data

d) Lack of tight configuration control between the obfuscated data and the keys used, so the wrong key versions were applied ✔

Correct answer: d | LO: AS-5.7.2 (K3) | Syllabus §5.7.2

Why: (d) — §5.7.2: **tight configuration control between the obfuscated data and keys used for the obfuscation is needed to ensure the correct versions of keys are used. Otherwise, the data can not be un-obfuscated for use.**

Q31. Derived from syllabus

Which topics should security training cover, according to the syllabus?

a) Only the password policy

b) The identification of social engineering attacks and insider threats, updated as new policies and threats arise, throughout the lifecycle ✔

c) Only secure coding for developers

d) Only the use of anti-malware tools

Correct answer: b | LO: AS-5.8.1 (K2) | Syllabus §5.8.1

Why: (b) — §5.8.1.

Q32. Derived from syllabus

A security tester injects a SQL command through a data entry field that lacks a field-level edit and views the customer database. The developer had attended secure-coding training last year.

What follow-up best tests and improves training effectiveness?

a) Dismiss the finding because the developer was trained

b) Remove the field from the application

c) Conclude the developer needs additional secure-coding training, and examine other developers' code to see whether the practice is widespread — if so, start a general process improvement initiative ✔

d) Run a password audit on the developer's account

Correct answer: c | LO: AS-5.8.2 (K3) | Syllabus §5.8.2

Why: (c) — the syllabus's second training-effectiveness example in §5.8.2.

Most Important Questions From This Chapter

*Ranking rationale: **four K2 and four K3** drawn from sixteen objectives. In the Sample Exam each mechanism appeared once. Weight your effort evenly, with a slight bias to the K3 (test) side, which is worth double.*

Rank	Topic / question focus	LO	Why it matters
1	IDS models and testing — Q5, Q22–Q24	AS-5.5.x	Most counter-intuitive answer (poison the profile); 2 points if K3.
2	Encryption testing families — Q3, Q16–Q18	AS-5.3.x	Longest test list; design / construction / configuration / ageing.
3	Obfuscation — Q7, Q28–Q30	AS-5.7.x	Brute force and decompilers; key control.
4	Hardening — Q1, Q9–Q11	AS-5.1.x	Audit + scanner + over-restriction.
5	AuthN vs AuthZ — Q2, Q12–Q15	AS-5.2.x	Clean K2 split; four test types.
6	Firewalls and zones — Q4, Q19–Q21	AS-5.4.x	Zone ≠ blocker; port scan / malformed / fragmentation.
7	Anti-malware — Q6, Q25–Q27	AS-5.6.x	Only known; EICAR; representative platform.
8	Training — Q8, Q31, Q32	AS-5.8.x	Humans weakest; three test examples.

Section 5 — Exam Tricks

5.1 Frequently tested topics (evidence-based)

Mechanism	K	Pts	Sample #	Format used officially
Hardening	**K3**	2	#25	Procedure to ensure hardening works → **monitor security reports and metrics**
AuthN/AuthZ	K2	1	#26	Key attribute of authentication → **user is legitimate**
Encryption	**K3**	2	#27	Gain confidence → **symmetric modes test**
Firewall / zones	K2	1	#28	Relationship → **sub-network = zone; firewall = traffic-monitoring software**
IDS	**K3**	2	#29	Most effective test → **malicious traffic to add intrusive specifications**
Anti-malware	K2	1	#30	Main disadvantage → **only known malware**
Obfuscation	**K3**	2	#31	Verify masking → **brute force**
Training	K2	1	#32	Weakest link → **human behavior**
Chapter 5 total	—	**12**	**#25–#32**	**15% of the points from the longest chapter**

The arithmetic

Chapter 5 takes **240 of 1,110 teaching minutes (22%)** but returns **12 of 80 points (15%)** — **3.0 points per hour**, tied lowest with Chapter 9. With **16 objectives for 8 questions**, roughly **half of what you learn here will not be examined on any given paper**. Study the **concept + weakness + test** triple for each mechanism (Table 3.1) and stop there.

5.2 Common distractors observed

- **Authorization described as authentication** (profile, rights, limited parts of the system).
- **Shared credentials** — each individual should have unique credentials.
- **"Random algorithms"** for randomness — the default random() is easy to crack.
- **Key-size numbers** not from the syllabus (256 bits) — the syllabus figure is **RSA ≥ 2048 bits (as of 2015)**.
- **The zone as the blocker** — the **firewall** blocks; zones define authorized flows.
- **Known-scenario testing of an IDS** — useful, but less effective than **malicious traffic that adds new intrusive specifications**.
- **Manual inspection** of masked data — not feasible at volume; **brute force** is the test.
- **Hiring a known hacker** — risk they don't tell you what they found.
- **Equilibrium** of hardening across components — critical areas may warrant more.

5.3 Keyword recognition (map words → answers)

If the stem says...	Think...
"reduce attack surface"	**Hardening**
"default passwords", "unnecessary services and ports"	**Hardening audit**
"who is the user / is it really them"	**Authentication**
"rights to do an action", "level of access"	**Authorization**
"123456", "pet name"	**Brute force / dictionary**
"../../", "/admin"	**Unauthorized URI / URL**
"modes", "key size", "self-signed"	**Encryption design** tests

If the stem says...	Think...
“random()”, “timing”, “power”	**Encryption construction** tests
“cipher order”, “downgrade”	**Encryption configuration** test
“DMZ”, “trust level”	**Network zones**
“port scan”, “malformed packets”, “fragmentation”	**Firewall** tests
“blacklist”, “signature”	**Negative** model
“white list”, “behaviour”, “regular expression”	**Positive** model
“EICAR”	**Anti-malware** functional test
“quarantined OS files”	Test anti-malware on a **representative platform** first
“masking”, “anonymization”	**Obfuscation**
“decompiler”, “unXOR”	**Code obfuscation** tests
“sticky notes”, “fake support call”	**Training effectiveness** tests

5.4 Typical traps

- **IDS vs firewall.** Firewall **stops** (looks outward); IDS **alerts** (analyzes suspicious intrusions).
- **Negative vs positive** — the names are counter-intuitive: **negative = blacklist = signatures**.
- **False negative** = an alert that **should have been reported but wasn’t**.
- **Phishing and pharming** are listed as **anti-malware detection targets**.
- **Obfuscation** is used **by attackers** too.
- **Code cannot fully protect itself against de-obfuscation**, because debugging can always be used.
- **Password audit tools** carry a risk — **passwords may be viewable to the tester**.

5.5 How to eliminate wrong answers

1. Identify the **mechanism** in the stem, then the **weakness** the syllabus names for it.
2. Strike options that **don’t test** the mechanism (they configure, buy, or remove it).
3. Strike options that **reduce** coverage of the weakness (remove WEP, disable heuristics).
4. Strike options that introduce **new risk** (hire a hacker, deploy without a pilot).
5. Of the remainder, choose the one using **syllabus test vocabulary**.

5.6 Scenario-solving shortcuts

Scenario shape	Shortcut
“Is hardening working?”	Audit configs + scanner + metrics; also check **over-restriction**
“Is login safe?”	Brute force/dictionary · SQLi · URL/URI · unpatched exploit
“Is encryption strong?”	Design (modes, keys, certs, replay) · construction (code review, fuzzing, timing, power) · configuration (TLS, cipher order) · ageing
“Is the firewall effective?”	Port scan → malformed/fuzzed packets → fragmentation; WAF → encoded payloads
“Is the IDS effective?”	Evasion for signatures; **malicious traffic to extend specifications** for behaviour
“Is anti-malware effective?”	EICAR; representative-platform pilot
“Is masking effective?”	Brute force/dictionary; decompilers for code
“Did training work?”	Social engineering call · desk sweep · password audit · tester SQLi · physical access

5.7 Common candidate mistakes

- Spending a week here. **Three evenings is enough** for 12 points.
- Bringing outside crypto knowledge into answers — answer **from the syllabus's lists**.
- Forgetting that hardening tests include checking it is **not too restrictive**.
- Assuming the official answer key is always internally consistent — see the Q27 and Q30 notes.

5.8 Mnemonics that carry the whole chapter

Mnemonic	Expands to
"Hard Auth Enc Fire IDS Mal Obf Train" ("**H**ow **A**re **E**very **F**ortress's **I**nner **M**oats **O**ften **T**ested?")	The eight mechanisms in syllabus order
D-C-C + Age	Encryption tests: **D**esign, **C**onstruction, **C**onfiguration, plus ageing
P-M-F ("Please Mind Fragments")	Firewall tests: **P**ort scan, **M**alformed packets/fuzzing, **F**ragmentation
"Negative = Naughty list"	Negative model = blacklist of known attacks
"Who? Really? May I?"	Login (who) · password (really) · authorization (may I)
"Known only"	Anti-malware's main disadvantage

5.9 Chapter 5 mind map

Branch	Leaves
Hardening	Design/construction/configuration · remove, patch, configure, remote logs · mechanisms to use · balance · audits, scanner, static analysis
AuthN/AuthZ	ID + token · OTP, certificates, SSO · rights and levels · brute force, SQLi, URI/URL, exploit
Encryption	Plain/cypher · at rest, in transit · symmetric/asymmetric · hashing, salting · DCC + ageing tests
Firewall	Host/network · zones of trust · filters · WAF · port scan, malformed, fragmentation, encoding
IDS	OSI layers · negative/positive · alerts vs blocks · evasion · false negatives · poisoning test
Anti-malware	Signatures + heuristics · phishing, pharming · evasion techniques · EICAR · pilot
Obfuscation	Masking, anonymization · techniques · encryption caveat · key control · brute force, decompilers
Training	Humans weakest · lifecycle-wide · social engineering, insiders · four effectiveness tests

Section 6 — Revision Notes

6.1 One-page chapter summary

Eight layers, none sufficient alone. **Hardening** shrinks the attack surface (remove, patch, configure, log remotely) — test with configuration audits, scanners and static analysis, and check it is not over-restrictive. **Authentication** asks who and whether truly; **authorization** asks what they may do — test with brute force, injection, URL tampering and unpatched exploits. **Encryption** turns plain into cypher text with algorithms and secrets — test design, construction, configuration and ageing.

Firewalls filter between zones of trust; **WAFs** add users and data patterns — test with port scans, malformed packets and fragmentation. **IDS** alert on policy violations via negative (signature) or positive (behaviour) models — test with evasion encodings and with malicious traffic to extend the behavioural specification. **Anti-malware** detects only what it knows — test with EICAR and a representative pilot. **Obfuscation** hides data and code — test with brute force and decompilers. **Training** addresses the weakest link — test with social engineering, desk sweeps and password audits.

6.2 Key definitions to memorize

Term	Definition
System hardening	Step-by-step attack-surface reduction by policy and layers of protection
Authentication	Verifying user identifier and token — who, and really who
Authorization	Rights of the authenticated user; level of access
Encryption	Plain → cypher text with algorithm and secrets
Firewall	Restricts access between a protected network and others
Network zone	Sub-network with a defined level of trust
IDS	Monitors OSI layers to detect policy violations and alert
Data obfuscation	Making data/code non-understandable for humans

6.3 High-priority concepts (study first)

6. Table 3.1 — mechanism → weakness → test (all four K3s).
7. Authentication vs authorization; firewall vs zone vs IDS (common K2s).
8. Negative vs positive IDS models.
9. Encryption test families.
10. Humans weakest link; anti-malware only knows the known.

6.4 Last-minute revision sheet

If you read nothing else

Hardening → audit defaults/services/ports/versions; scanner; static analysis; not too restrictive.

AuthN = legitimate user. AuthZ = rights. Tests: brute force, SQLi, ../../ and /admin, unpatched exploit.

Encryption → **symmetric modes**, RSA ≥ 2048, certificates, replay; no random(); timing/power; TLS cipher order; ageing.

Zone = sub-network with trust level; firewall blocks; WAF filters users and patterns. Tests: port scan, malformed/fuzz, fragmentation.

IDS negative = signature = blacklist; positive = behaviour = white list. Test: evasion; **malicious traffic adds intrusive specs**.

Anti-malware: **known only**; EICAR; pilot on representative platform.

Obfuscation: brute force/dictionary; decompilers; key control.

Training: **humans weakest**; fake support call, sticky notes, password audit.

6.5 Memorization checklist

✓	I can…
☐	explain hardening and list five removals and three audits
☐	separate authentication from authorization and list four tests
☐	list encryption tests under design, construction, configuration and ageing
☐	describe zones, firewall filters, WAF filters and three firewall tests
☐	contrast negative and positive IDS models and name their tests
☐	state anti-malware's weakness, EICAR, and the pilot rule
☐	list obfuscation techniques and tests
☐	give three training-effectiveness tests

6.6 Exam-day quick review

- Expect **one question per mechanism**, alternating K2 and K3.
- **Find the weakness; the test exploits it.**
- Answer encryption key-size questions **from the syllabus (2048-bit RSA)**.